OPTIMALITY OF VALUE FUNCTIONS AND POLICIES

In the context of Markov Decision Processes (MDPs), the goal is to find an optimal policy $\pi*$ that maximizes the expected cumulative reward starting from any state. The concepts of optimal value functions and optimal policies are closely related, and they form the foundation for solving MDPs.

Optimal Value Functions

Optimal State-Value Function (V∗(s))

The **optimal state-value function** is defined as the maximum expected cumulative reward achievable from state s , following any policy:

$$V^*(s) = \max_{\pi} V_\pi(s)$$

This represents the best possible value for state s over all policies.

Optimal Action-Value Function (Q^*(s , a))

The optimal action-value function is defined as the maximum expected cumulative reward achievable from state s, taking action a, and then following the best possible policy:

$$Q^*(s,a) = \max_{\pi} Q_\pi(s,a)$$

This represents the best possible value for state-action pair (s,a) over all policies.

Relationship Between V^* and Q^*

The optimal state-value function can be expressed in terms of the optimal action-value function:

$$V^*(s) = \max_{a \in A} Q^*(s,a)$$

Conversely, Q∗(s,a) relates to V∗ as:

$$Q^*(s,a) = R(s,a) + \gamma \sum_{s' \in S} P(s'|s,a) V^*(s')$$

Bellman Optimality Equations

The **Bellman Optimality Equations** describe the recursive relationships for V∗ and Q∗.

State-Value Function Optimality

$$V^*(s) = \max_{a \in A} \left[R(s,a) + \gamma \sum_{s' \in S} P(s'|s,a) V^*(s') \right]$$

This equation states that the value of the optimal policy at state s is the maximum expected return over all possible actions.

Action-Value Function Optimality

$$Q^*(s,a) = R(s,a) + \gamma \sum_{s' \in S} P(s'|s,a) \max_{a' \in A} Q^*(s',a')$$

This equation states that the value of the optimal policy for a state-action pair is the immediate reward plus the discounted value of the next state, assuming optimal actions thereafter.

Optimal Policies

An **optimal policy** $\pi*$ is one that achieves the optimal value functions $V*$ and $Q*$. Formally:

$$\pi^*(s) = \arg\max_{a \in A} Q^*(s,a)$$

Properties of Optimal Policies

1. **Deterministic**: If an MDP has a unique optimal policy, it is deterministic (chooses the same action for a given state).
2. **Equivalence**: All optimal policies achieve the same value functions $V*(s)$ and $Q*(s ,a)$.
3. **Greediness**: An optimal policy is "greedy" with respect to the optimal value function:

Key Insights

- **Optimal value functions** encode the maximum rewards achievable from states or actions.
- **Optimal policies** are derived by being greedy with respect to these value functions.
- The Bellman Optimality Equations serve as the foundation for many algorithms used to compute optimal policies, including dynamic programming, Monte Carlo methods, and temporal-difference learning.

POLICY EVALUATION

Policy evaluation is the process of determining the **value function** of a given policy (π) in a reinforcement learning setting. It provides a quantitative measure of how good it is to follow a particular policy, by estimating either the **state-value function** ($V\pi(s)$) or the **action-value function** ($Q\pi(s,a)$).

Key Components of Policy Evaluation

Policy (π)

- A mapping from states to actions.
- Determines the probability of taking each action in a given state.

State-Value Function ($V^{\pi}(s)$):

Represents the expected return when starting in state sss and following policy π

$$V^{\pi}(s) = \mathbb{E}_{\pi}\left[\sum_{t=0}^{\infty} \gamma^{t} r_{t+1} \mid s_0 = s\right]$$

Where:

- r_{t+1}: Reward at time $t+1$.
- γ: Discount factor ($0 \leq \gamma < 1$).

Action-Value Function ($Q^{\pi}(s,a)$):

Represents the expected return for taking action a in state s, and then following policy π

$$Q^{\pi}(s,a) = \mathbb{E}_{\pi}\left[\sum_{t=0}^{\infty} \gamma^{t} r_{t+1} \mid s_0 = s, a_0 = a\right]$$

Policy Evaluation Procedure

Policy evaluation aims to compute $V^{\pi}(s)$ or $Q^{\pi}(s,a)$ for all states and actions under a given policy. The computation is typically performed using **iterative methods** based on the Bellman Expectation Equation.

Bellman Expectation Equation

For a given policy π, the Bellman Expectation Equation for $V\pi(s)$ is:

$$V^\pi(s) = \sum_a \pi(a \mid s) \sum_{s'} P(s' \mid s, a) \left[R(s, a, s') + \gamma V^\pi(s')\right]$$

Where:

- P(s′|s,a): Probability of transitioning to state s′ after taking action a in state s.
- R(s,a,s′): Reward received when transitioning from s to s′ via a.

Iterative Policy Evaluation

The Bellman Expectation Equation can be solved iteratively:

1. Initialize $V^\pi(s)$ arbitrarily (e.g., $V^\pi(s) = 0$).
2. Repeat until convergence:
$$V^\pi(s) \leftarrow \sum_a \pi(a \mid s) \sum_{s'} P(s' \mid s, a) \left[R(s, a, s') + \gamma V^\pi(s')\right]$$

Convergence

The iterative approach converges to the true $V^\pi(s)$ if $0 \leq \gamma < 1$.

Example

Gridworld Example

- A 3x3 grid where:
  - States represent grid cells.
  - Actions: Up, Down, Left, Right.
  - Rewards: -1 per step, $+10$ for reaching the goal.

Policy (π)

- Assume a uniform random policy:
  - Each action is chosen with equal probability: $\pi(a \mid s) = 0.25$.

Policy Evaluation

- Initialize $V^\pi(s) = 0$ for all states.
- Use the Bellman Expectation Equation iteratively to update $V^\pi(s)$.

Applications of Policy Evaluation

1. **Policy Improvement**:
   - Evaluate a policy to identify areas for improvement.
   - Used in algorithms like **Policy Iteration**.
2. **Actor-Critic Methods**:
   - The "critic" estimates $V\pi(s)$ to guide the "actor" in policy updates.
3. **Model-Based Methods**:
   - Policy evaluation is used to compute value functions based on a known model of the environment.

Summary

Policy evaluation computes the value of a policy by estimating the expected return for each state (or state-action pair). It forms the foundation of many reinforcement learning methods, including policy iteration, actor-critic algorithms, and dynamic programming techniques. By accurately estimating the value function, policy evaluation provides critical feedback for policy optimization.